

CENTRO UNIVERSITÁRIO DE BRASÍLIA – CEUB
[CURSO]

Integrantes

Jhonathan Magalhães da Cruz

RA: 22502908

Fellipe de Castro Oliveira

RA: 2250324

Repositórios

[Front-end](#)

[Back-end](#)

NAVALHA

PLATAFORMA SAAS MULTI-TENANT PARA GESTÃO DE BARBEARIAS

Documentação técnica de arquitetura full stack e integração frontend–backend

BRASÍLIA

2026

[NOME COMPLETO DO(S) AUTOR(ES)]

NAVALHA

PLATAFORMA SAAS MULTI-TENANT PARA GESTÃO DE BARBEARIAS

Documentação técnica de arquitetura full stack e integração frontend–backend

Trabalho apresentado ao Centro
Universitário de Brasília (CEUB) como
requisito parcial para avaliação na
disciplina [DISCIPLINA], do curso de
[CURSO].

Orientador(a): Prof.(a) [NOME DO(A)
PROFESSOR(A)].

BRASÍLIA

2026

RESUMO

Este trabalho apresenta a documentação técnica do Navalha, uma plataforma de software como serviço (SaaS) multi-tenant voltada à gestão de barbearias, contemplando agendamento, recepção, controle de caixa, painel administrativo do proprietário, aplicativo do barbeiro e área do cliente. O sistema adota arquitetura cliente-servidor desacoplada, com frontend desenvolvido em Next.js e React, comunicando-se por meio de uma API REST construída em Spring Boot e persistência em banco de dados relacional PostgreSQL. A documentação descreve a arquitetura geral, as camadas de frontend, backend e persistência, os mecanismos de autenticação por JSON Web Token (JWT), o controle de acesso baseado em papéis (RBAC), a política de compartilhamento de recursos entre origens (CORS) e a estratégia de implantação em ambientes de nuvem distintos para cada camada. São também relatados os principais problemas identificados durante a integração entre frontend e backend, as soluções aplicadas e o grau de maturidade de cada módulo, distinguindo as funcionalidades plenamente operacionais das que permanecem planejadas para fases futuras. Conclui-se que o sistema cumpre os objetivos do produto mínimo viável (MVP), validando a arquitetura proposta e o fluxo central de operação com dados reais.

Palavras-chave: Engenharia de software. SaaS multi-tenant. Arquitetura full stack. API REST. Next.js. Spring Boot. PostgreSQL.

1 INTRODUÇÃO

O Navalha é uma plataforma de software como serviço (SaaS) multi-tenant destinada à gestão de barbearias, projetada para o mercado brasileiro. A solução reúne, em um único produto, os fluxos de agendamento, recepção, controle de caixa, gestão da equipe, painel administrativo do proprietário, aplicativo do barbeiro e área do cliente, permitindo que múltiplos estabelecimentos operem de forma isolada sobre a mesma infraestrutura.

O produto está organizado em dois repositórios independentes: um para o frontend e outro para o backend. O frontend foi construído sobre um contrato de API REST documentado, podendo operar tanto em modo simulado (com dados fictícios interceptados no navegador) quanto contra a API real. O backend implementa a primeira fase (MVP) do contrato, restando áreas ainda não implementadas no servidor que são tratadas, no cliente, por mecanismos de degradação controlada.

Este documento tem por objetivo descrever, de forma estruturada, a arquitetura do sistema, suas camadas, os mecanismos de segurança, a estratégia de implantação, o estado da integração entre frontend e backend e os problemas técnicos enfrentados, bem como as soluções adotadas. Pretende-se, com isso, registrar as decisões de projeto e o grau de maturidade alcançado por cada módulo.

2 ARQUITETURA DO SISTEMA

2.1 Visão geral

O sistema adota arquitetura cliente-servidor desacoplada, na qual a camada de apresentação (frontend) é independente da camada de aplicação (backend), comunicando-se exclusivamente por requisições HTTP sobre um contrato REST. A persistência é realizada em banco de dados relacional. Cada camada é implantada em um provedor de nuvem distinto, conforme sintetizado no Quadro 1.

Quadro 1 — Camadas, tecnologias e hospedagem

Camada	Tecnologias	Hospedagem
Frontend	Next.js 15, React 19, TypeScript, Tailwind, TanStack Query	Netlify (navalha.netlify.app)
Backend	Spring Boot 3.3.5, Java 17, JPA, Flyway	Render (onrender.com)
Banco de dados	PostgreSQL 16 (Supabase em produção; Docker em desenvolvimento)	Supabase / contêiner local

Fonte: elaborado pelos autores (2026).

A Figura 1 apresenta a visão geral da arquitetura, evidenciando o fluxo entre o navegador, a aplicação de frontend, a API de backend e o banco de dados, bem como a alternância entre o modo simulado e o modo de produção, controlada por variável de ambiente.



camadas e fluxo entre os repositórios

2.2 Fluxo de execução em produção

Em produção, o fluxo típico inicia-se na tela de autenticação, na qual as credenciais são enviadas ao endpoint de login. Em caso de sucesso, os tokens de acesso e de atualização são armazenados no navegador e a aplicação consulta o endpoint de identificação do usuário, obtendo seu papel e suas unidades. A partir do papel, o controle de rotas direciona o usuário à área correspondente — por exemplo, o proprietário é encaminhado ao painel administrativo. As telas, então, consomem os serviços de API responsáveis por cada domínio funcional.

3 CAMADA DE FRONTEND

3.1 Tecnologias e organização

O frontend é construído com Next.js 15 e React 19, em TypeScript, utilizando a biblioteca TanStack Query para gerenciamento de cache e revalidação de dados, React Hook Form com Zod para formulários, e a biblioteca Recharts para a renderização de gráficos no painel do proprietário. Os dados simulados, quando ativados, são providos por um service worker de interceptação de requisições. A suíte de testes utiliza Vitest e Testing Library.

O código é organizado por domínio de negócio: um diretório concentra as funcionalidades por papel (proprietário, recepção, barbeiro, entre outros); outro reúne componentes de interface, integração com a API, hooks e tipos compartilhados; e um terceiro contém as rotas, mantidas enxutas segundo o roteamento por diretórios do framework.

3.2 Rotas e papéis de acesso

A aplicação possui aproximadamente vinte e seis páginas, distribuídas conforme o papel do usuário. O Quadro 2 resume as principais rotas e suas finalidades.

Quadro 2 — Principais rotas do frontend por papel

Papel	Rotas	Finalidade
Público	/, /marketplace, /u/[slug]/agendar	Página inicial, descoberta e agendamento sem login
Autenticação	/entrar, /cadastro	Login e cadastro
Proprietário/Gerente	/dono e subrotas	Indicadores, gráficos, equipe e comissão
Recepção	/recepcao e subrotas	Operação do dia, caixa e fila
Barbeiro	/barbeiro e subrotas	Agenda, clientes, comissão e portfólio
Cliente	/cliente e subrotas	Agendamento e programa de fidelidade
Onboarding	/onboarding	Assistente de configuração inicial

Fonte: elaborado pelos autores (2026).

3.3 Autenticação no cliente

O token de acesso é mantido tanto no armazenamento local do navegador quanto em um cookie, este último necessário para que o controle de rotas (executado no servidor de borda do framework) reconheça a sessão durante a navegação entre páginas. A renovação do token ocorre automaticamente diante de

respostas de não autorizado. O controle de rotas exige a presença de um token válido nas áreas restritas e, após o login, redireciona o usuário conforme seu papel.

3.4 Operação em modo simulado e modo real

Uma variável de ambiente determina se a aplicação opera com dados simulados ou contra a API real. No modo simulado, as requisições são interceptadas no navegador e respondidas com dados fictícios, dispensando o backend. No modo real, as requisições são dirigidas ao servidor, sendo obrigatória a presença de um token de autenticação válido. Para a operação contra a API real, a variável que define o endereço-base da API deve apontar para o backend hospedado; na ausência dessa configuração, a aplicação utiliza um endereço local padrão, o que provoca falhas de conexão em produção.

4 CAMADA DE BACKEND

4.1 Tecnologias

O backend é desenvolvido em Java 17 com Spring Boot 3.3.5, utilizando Spring Security para autenticação e autorização, JSON Web Tokens para a sessão, Spring Data JPA para o mapeamento objeto-relacional e Flyway para o versionamento do esquema do banco. A serialização JSON adota a convenção `snake_case`. O prefixo global das rotas é `/api/v1`.

4.2 Módulos e endpoints

O backend implementa os módulos que sustentam o núcleo do MVP. O Quadro 3 relaciona os módulos e seus principais endpoints.

Quadro 3 — Módulos e principais endpoints do backend

Módulo	Principais endpoints
Autenticação	POST <code>/auth/signup</code> , <code>/login</code> , <code>/refresh</code> ; GET <code>/me</code>
Tenants	GET/PATCH <code>/tenant</code> ; GET/POST/PATCH <code>/units</code>
Onboarding	GET <code>/onboarding</code> ; PATCH <code>/onboarding/steps/{n}</code> ; POST <code>/publish</code>
Catálogo	CRUD de <code>/services</code>
Equipe	<code>/barbers</code> , jornadas e bloqueios
Agendamentos	<code>/availability</code> ; CRUD de <code>/appointments</code> ; confirmação e cancelamento
Lista de espera	GET/POST <code>/waitlist</code>
API pública	<code>/public/u/{slug}</code> , agendamento e <code>/public/marketplace</code>

Módulo	Principais endpoints
Clientes	CRUD parcial, histórico e fidelidade
Caixa	Sessões de caixa; POST /payments e intenção de PIX
Métricas	GET /metrics/overview, /revenue-series, /barber-ranking

Fonte: elaborado pelos autores (2026).

4.3 Módulo de métricas

O módulo de métricas fornece os indicadores consumidos pelo painel do proprietário. Os três endpoints realizam agregações sobre as tabelas de agendamentos e de perfis de barbeiro, considerando como receita os agendamentos em estados relevantes. O acesso é restrito aos papéis de proprietário e gerente. O Quadro 4 descreve os endpoints.

Quadro 4 — Endpoints do módulo de métricas

Endpoint	Parâmetros	Resposta
/metrics/overview	unit_id; period	Receita, variação, ocupação, ticket médio e taxa de ausência
/metrics/revenue-series	unit_id; granularity	Série de faturamento por período
/metrics/barber-ranking	unit_id; period	Ranking de barbeiros por receita

Fonte: elaborado pelos autores (2026).

4.4 Funcionalidades pendentes

Diversas áreas previstas no contrato de API ainda não foram implementadas no servidor, permanecendo como trabalhos de fases posteriores. O Quadro 5 apresenta as principais pendências.

Quadro 5 — Funcionalidades previstas e ainda não implementadas no backend

Área	Situação
Comissão	Não implementado
WhatsApp e webhooks de mensageria	Não implementado
Webhooks de pagamento e recorrência	Não implementado
Relatórios financeiros (DRE, fluxo de caixa)	Não implementado
Estoque de produtos	Não implementado
Portfólio e avaliações	Não implementado
Comunicação em tempo real (WebSocket)	Não implementado
Clube e planos de assinatura	Não implementado
Anti-ausência avançado e overbooking	Parcial (apenas leitura do risco)

Fonte: elaborado pelos autores (2026).

5 CAMADA DE PERSISTÊNCIA

5.1 Esquema do banco de dados

O esquema, versionado por migração, é composto por dezesseis tabelas, entre as quais tenants, unidades, usuários, perfis de barbeiro, serviços, clientes, agendamentos, itens de agendamento, jornadas de trabalho, bloqueios de horário, sessões de caixa, pagamentos, cartões de fidelidade e estado de onboarding. Adotam-se como convenções chaves primárias do tipo texto, valores monetários armazenados em centavos inteiros, marcações de tempo com fuso horário e enumerações representadas por texto com restrições de verificação.

5.2 Segurança em nível de linha

Uma segunda migração habilita a segurança em nível de linha (Row-Level Security), preparando o banco para o isolamento de dados por tenant em cenários de acesso direto. A aplicação, contudo, conecta-se com privilégios de proprietário do banco, de modo que o isolamento, em produção pela aplicação, é garantido pela própria camada de serviço.

5.3 Carga de dados de demonstração

Na inicialização, um carregador de dados popula a base com informações de demonstração, de forma idempotente — a carga só é executada caso o tenant de demonstração ainda não exista. São criados um tenant, unidades, usuários para cada papel, agendamentos do dia e uma sessão de caixa aberta, viabilizando a demonstração imediata do sistema.

6 SEGURANÇA, AUTENTICAÇÃO E CORS

6.1 Autenticação por JWT

A autenticação é stateless, baseada em JSON Web Tokens. São emitidos um token de acesso, de curta duração, e um token de atualização, de duração mais longa, assinados com segredos distintos. Os tokens carregam, entre suas reivindicações, o identificador do usuário, o identificador do tenant, o papel e o tipo. O provedor de autenticação padrão valida as senhas com o algoritmo BCrypt.

6.2 Autorização e controle de acesso

O controle de acesso baseado em papéis é aplicado de forma granular nos controladores, por meio de anotações de pré-autorização habilitadas na configuração de segurança. As rotas públicas — verificação de integridade, autenticação e endpoints públicos — dispensam token; as demais exigem um token válido. No frontend, o controle de rotas reforça a separação por papel, impedindo o acesso de um perfil a áreas destinadas a outro.

6.3 Política de CORS

A política de compartilhamento de recursos entre origens é configurada no backend e integrada à cadeia de filtros de segurança, de modo que as requisições de verificação prévia (preflight) sejam corretamente processadas antes da autenticação. A origem do frontend hospedado deve constar na lista de origens permitidas do backend; caso contrário, o navegador bloqueia as chamadas. Verificou-se que, com a configuração correta da variável de origens, as requisições provenientes do domínio de produção são atendidas com os cabeçalhos apropriados.

7 IMPLANTAÇÃO E AMBIENTES

O sistema é implantado em três contextos. No ambiente de desenvolvimento, o frontend e o backend são executados localmente, com o banco de dados em contêiner. Em produção, o frontend é publicado na plataforma Netlify e o backend na plataforma Render, cujo plano gratuito implica suspensão por inatividade e, conseqüentemente, um tempo de reativação inicial (cold start). O banco de dados de produção é provido pela plataforma Supabase.

A correta operação em produção depende de uma lista de verificação: a configuração, no frontend, do endereço-base da API apontando para o backend; a inclusão, no backend, do domínio do frontend entre as origens permitidas; e a definição das variáveis de segredo, de conexão com o banco e de carga de dados. Toda alteração no backend exige nova publicação para que produza efeito em produção, dado que a aplicação é compilada.

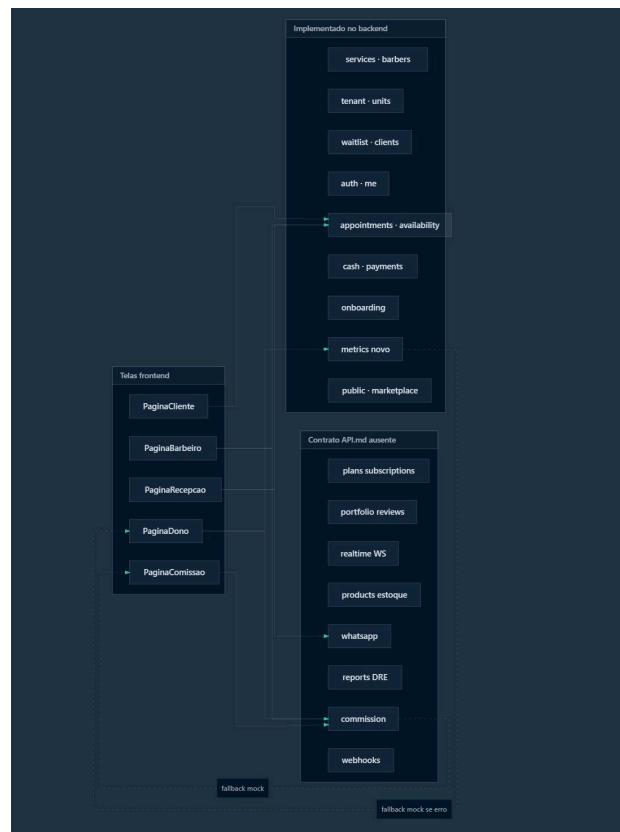
8 INTEGRAÇÃO ENTRE FRONTEND E BACKEND

Em produção, com a API real, encontram-se plenamente funcionais a autenticação e a renovação de sessão, a identificação do usuário, o marketplace público, o perfil público e o agendamento sem login, a gestão de tenant, unidades, serviços e barbeiros, os agendamentos e a disponibilidade, os clientes e a fidelidade básica, o caixa e os pagamentos, o onboarding e as métricas do painel do proprietário.

Algumas telas, entretanto, consomem endpoints ainda inexistentes no servidor — como comissão, mensageria, estoque, portfólio e clube de assinaturas. Para essas situações, o frontend foi projetado com resiliência: o painel do proprietário, por exemplo, recorre a dados de exemplo quando a consulta às métricas falha, sinalizando ao usuário, por meio de um aviso, se os dados exibidos são reais ou de demonstração. Dessa forma, a indisponibilidade de um endpoint não compromete a renderização da tela.

A Figura 3 ilustra o mapeamento entre as telas do frontend e os endpoints do backend, distinguindo os endpoints já implementados no servidor daqueles ainda ausentes no contrato de API, para os quais o frontend recorre ao mecanismo de dados simulados.

Figura 3 — Mapeamento entre telas do frontend e endpoints do backend



Fonte: elaborado pelos autores (2026).

9 PROBLEMAS IDENTIFICADOS E SOLUÇÕES

Durante a integração entre frontend e backend foram diagnosticados e tratados diversos problemas. O Quadro 6 sintetiza os principais sintomas, suas causas e a situação de cada um.

Quadro 6 — Problemas identificados, causas e situação

Sintoma	Causa identificada	Situação
Falha de conexão no frontend hospedado	Endereço-base da API ausente ou incorreto na publicação	Corrigido por configuração
Não autorizado após o login	Papel extraído de forma incompatível com o token real e cookie de sessão	Corrigido no controle de rotas
Erro interno em endpoints de métricas	Endpoints inexistentes no servidor	Endpoints implementados
Erro interno em rota inexistente	Tratador genérico converte ausência de rota em erro interno	Melhoria futura: retornar não encontrado
Reativação lenta do backend	Suspensão por inatividade no plano gratuito	Comportamento esperado do plano

Fonte: elaborado pelos autores (2026).

10 CONSIDERAÇÕES FINAIS

A documentação evidencia que o Navalha atinge os objetivos de um produto mínimo viável, validando a arquitetura full stack proposta e o fluxo central de operação com dados reais. As camadas de autenticação e multi-tenancy, bem como o núcleo de agendamento e reserva, apresentam alta maturidade; o caixa e o painel de métricas situam-se em maturidade intermediária; e as funcionalidades de comissão, mensageria, estoque, portfólio e clube permanecem em estágio inicial, com interface pronta e implementação de servidor planejada para fases futuras. O Quadro 7 resume essa avaliação.

Quadro 7 — Maturidade por camada

Camada	Maturidade	Observação
Autenticação e multi-tenancy	Alta	JWT, carga de demonstração e isolamento por tenant
Agendamento e reserva	Alta	Núcleo do MVP
Caixa e pagamentos	Média	Intenção de PIX ainda é simulada
Painel do proprietário (métricas)	Média-alta	Módulo recém-implementado

Camada	Maturidade	Observação
Comissão e financeiro	Baixa	Apenas interface; servidor ausente
Mensageria e tempo real	Baixa	Apenas interface; servidor ausente
Estoque, portfólio e clube	Baixa	Apenas interface; servidor ausente

Fonte: elaborado pelos autores (2026).

Como trabalhos futuros, recomenda-se a implementação, no backend, dos módulos de comissão, mensageria, estoque, portfólio e clube de assinaturas; a evolução do tratamento de erros para a distinção entre rota inexistente e erro

Desenvolvimento e Inteligência Artificial

Este projeto foi planejado e construído com o auxílio de modelos de Inteligência Artificial, divididos estrategicamente em duas frentes de trabalho:

Planejamento e Engenharia de Código

A arquitetura, a lógica do sistema e o planejamento das funcionalidades foram desenvolvidos em colaboração com o Claude 3.5 Sonnet.

Escopo de atuação: Estruturação de pastas, escrita de código, refatoração e resolução de problemas de lógica de programação.

Geração de Imagens e Ativos Visuais

A identidade visual e as ilustrações do projeto foram geradas utilizando o Google Gemini.

* **Escopo de atuação:** Criação de imagens conceituais, texturas e elementos gráficos que compõem a interface e a ambientação do projeto.

Fluxo de Trabalho

interno; e a substituição da intenção de pagamento simulada por integração efetiva com adquirentes. Tais evoluções permitirão que as telas hoje sustentadas por dados de exemplo passem a operar integralmente com dados reais.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6023: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

META PLATFORMS. React Documentation. [S. l.], 2025. Disponível em: <https://react.dev>. Acesso em: 25 jun. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 16 Documentation. [S. l.], 2024. Disponível em: <https://www.postgresql.org/docs/16/>. Acesso em: 25 jun. 2026.

SPRING. Spring Boot Reference Documentation. [S. l.], 2024. Disponível em: <https://docs.spring.io/spring-boot/>. Acesso em: 25 jun. 2026.

TANSTACK. TanStack Query Documentation. [S. l.], 2025. Disponível em: <https://tanstack.com/query>. Acesso em: 25 jun. 2026.

VERCEL. Next.js Documentation. [S. l.], 2025. Disponível em: <https://nextjs.org/docs>. Acesso em: 25 jun. 2026.